

Лабораторная работа №1. Настройка Visual Studio Code и работа с Git в терминале

Особенности Visual Studio Code

VS Code — это:

- Легковесный редактор.
- Гибко настраивается за счёт расширений.
- Кроссплатформенный.
- Имеет встроенную интеграцию с Git.
- Поддерживает работу через терминал.

Наша итоговая цель — создание полноценного **full-stack-приложения**, поэтому мы будем работать в одной среде, которая подходит и для **frontend**, и для **backend**, не перегружая систему тяжёлыми IDE.

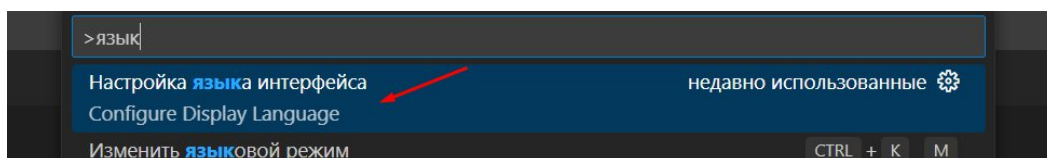
Ранее вы работали в **Visual Studio 2022**, знаете **C#** и основы **Git**, но практически не использовали **терминал**. С этого занятия терминал станет обязательной частью работы.

Шаг 1. Первичная настройка VS Code

1. Запустите **Visual Studio Code**.

2. Если VS Code открылся на русском языке — переключите интерфейс на английский:

- Нажмите **Ctrl + Shift + P**
- Введите **язык**
- Выберите **настройки языка интерфейса**



- Установите **en**



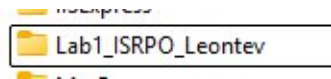
- Перезапустите приложение

3. Масштаб интерфейса:

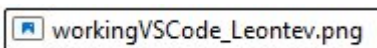
- Увеличить: **Ctrl + =**
- Уменьшить: **Ctrl + -**

Практическое задание 1

Создайте папку с именем **Lab1_ISRPO_FIO** (где укажите ваши фамилии) в папке *Работы студентов* или в *Документах*:

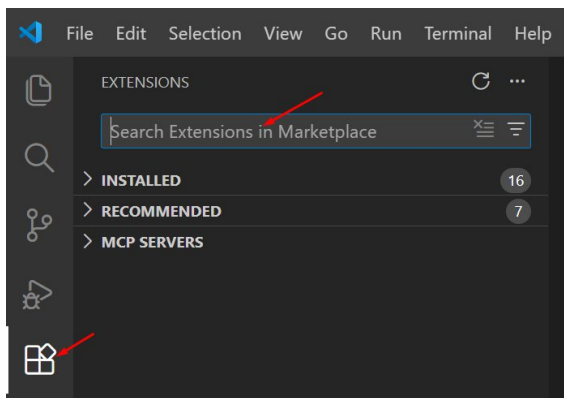


1. Запустите VS Code.
2. Переключите язык интерфейса на английский.
3. Настройте комфортный масштаб интерфейса.
4. Сделайте **скриншот** интерфейса после настройки, назовите его **workingVSCode_FIO.png** и сохраните его в папке с лабораторной. Для создания скриншотов можно использовать встроенные средства Windows или более продвинутые инструменты, например **Flameshot** (возможность редактирования, добавление подписей и т.д.)

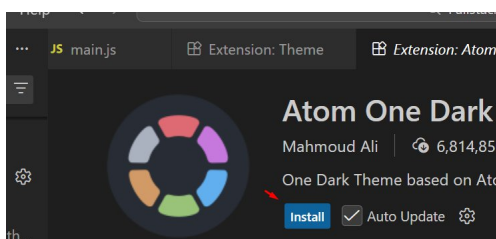


Шаг 2. Настройка внешнего вида (темы)

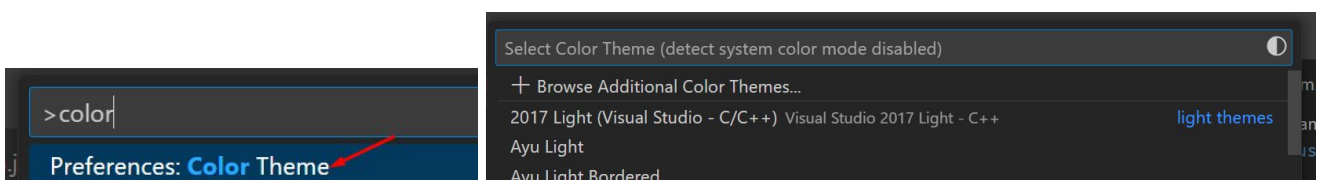
1. Перейдите во вкладку **Extensions**.
2. Введите в поиске название темы.



3. Нажмите **Install**



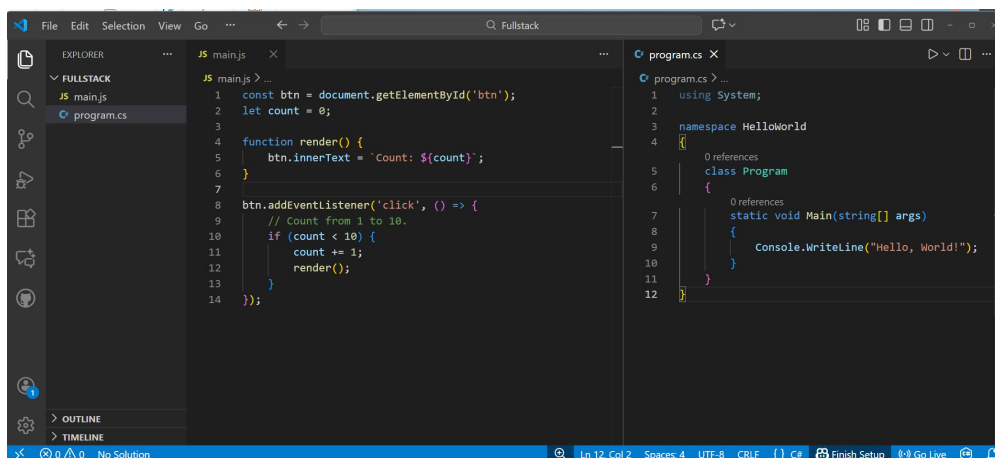
4. После установки нажмите **Ctrl + Shift + P**, введите **Color Theme** и выберите тему.



Ниже подобраны одни из наиболее интересных тем, которые часто используются в разработке. Ваша задача выбрать одну из них для себя.

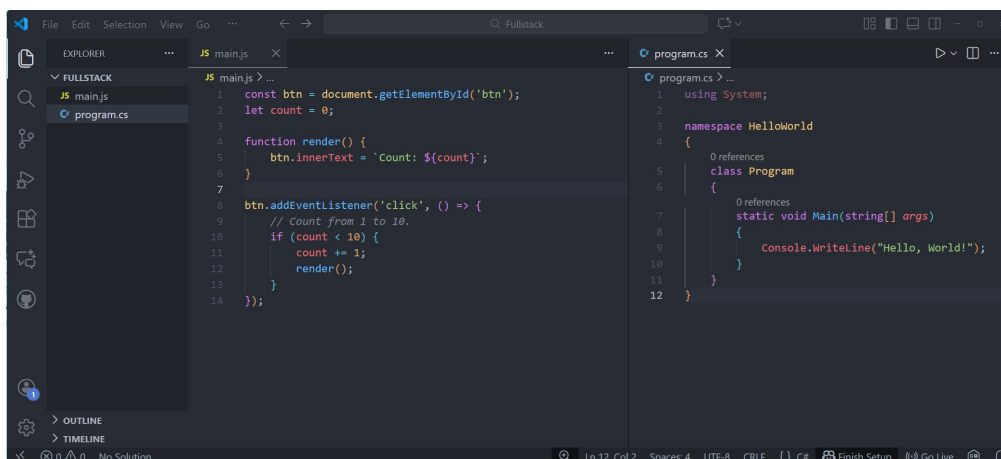
1. Visual Studio 2019

Используемая тема по умолчанию.



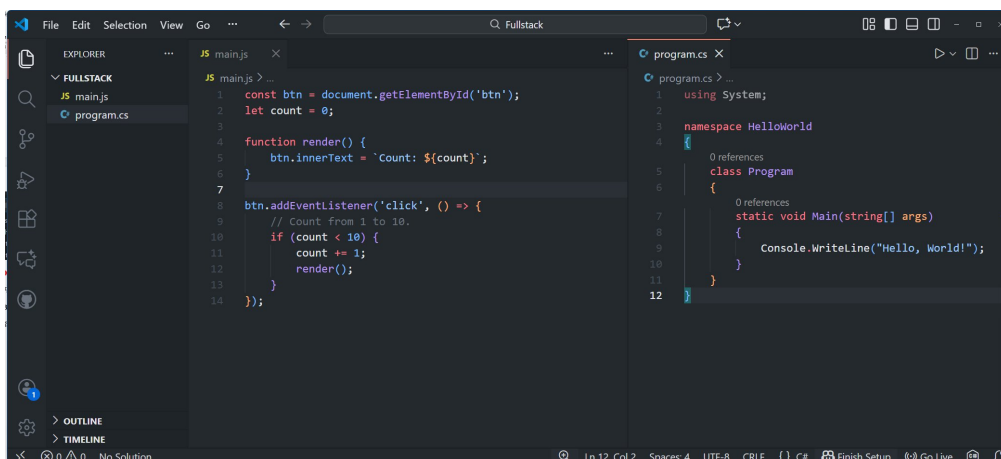
2. One Dark Pro

Знаменитая тема от Atom. Славится цветовыми сочетаниями и приятным стилем. Подойдёт для любителей тёмного режима.



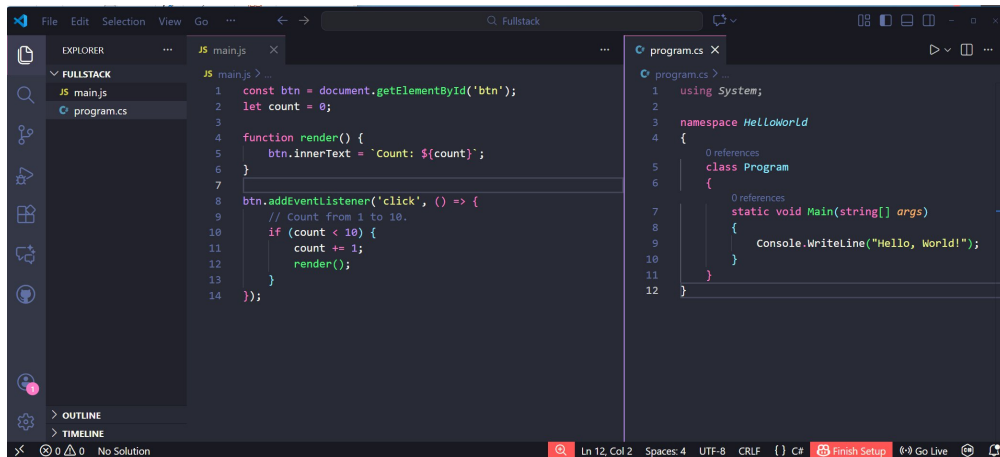
3. GitHub Theme

Для любителей классических тем **GitHub**. Доступна в девяти версиях.



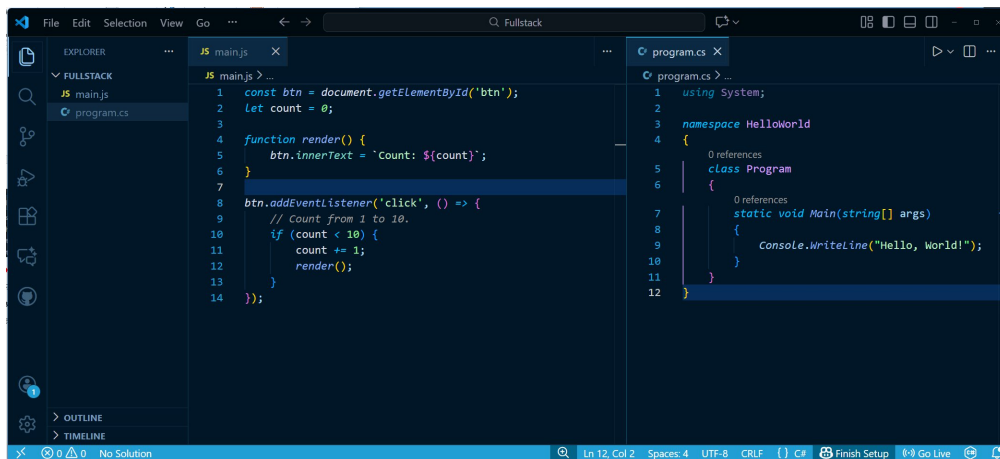
4. Dracula Official

Ещё один вариант тёмной темы, которую часто используют дизайнеры. Доступна в классической тёмной и мягкой версиях с менее контрастными цветами.



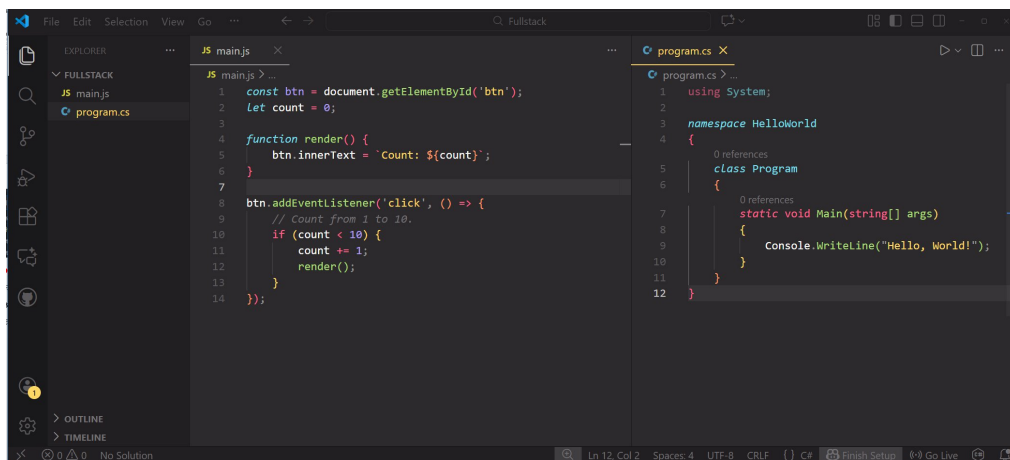
5. Winter is Coming Theme

Для любителей синего цвета. Доступна в тёмной, светлой и версии без курсива.



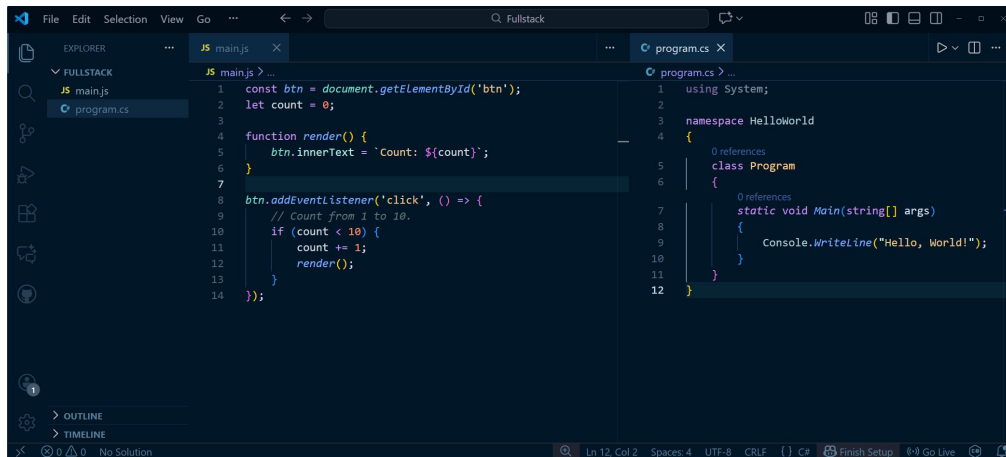
6. Monokai Pro

Цель темы — фокусировка только на коде. Доступна в шести версиях.



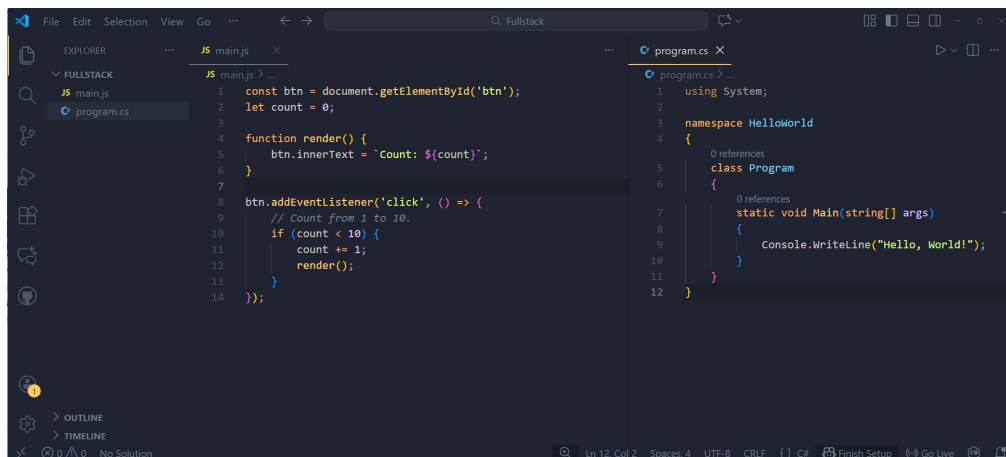
7. Night Owl

Тема для ночного и дневного использования, где подобран идеальный контраст цвета. Удобно, когда вы любите поработать ночью. Доступна в тёмной, светлой и версии без курсива.



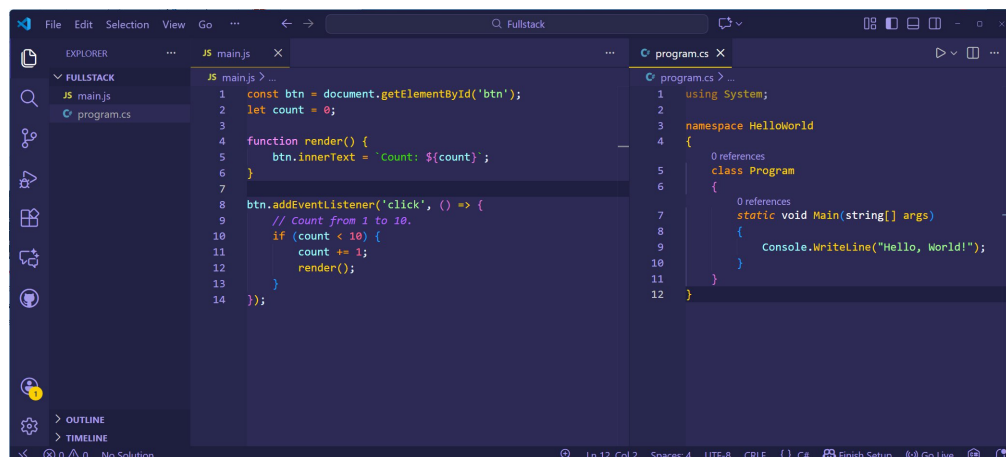
8. Ayu

Простая тема с яркими цветами. Доступна в шести версиях — светлой, тёмной, с рамкой и версии mirage с серым фоном.



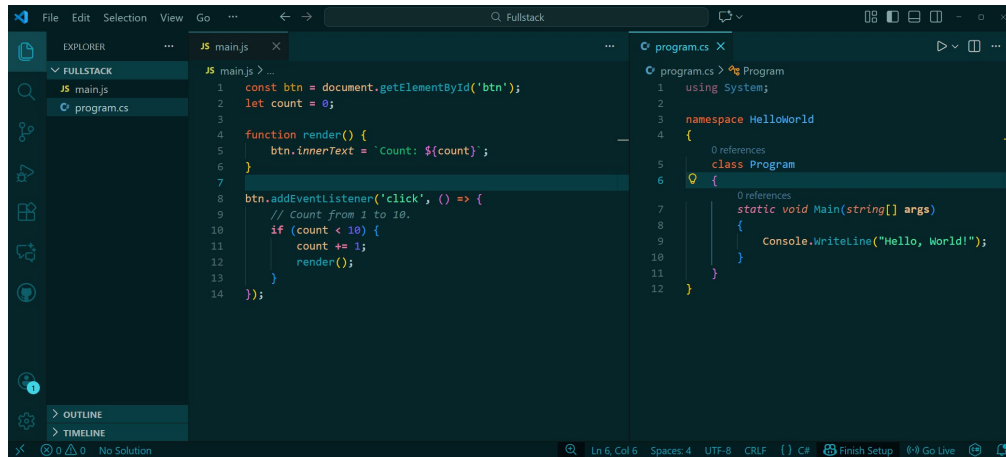
9. Shades of Purple

Тема с яркими оттенками фиолетового. Благодаря хорошо подобранным цветовым выделениям, удобно ориентироваться в коде.



10. Noctis

Коллекция светлых и тёмных тем с хорошим сочетанием тёплых и холодных оттенков. Доступна аж в 11 версиях. Точно найдёте на свой вкус.



После экспериментов **удалите неиспользуемые темы.**

В дальнейшем в примерах используется тема **One Dark Pro**.

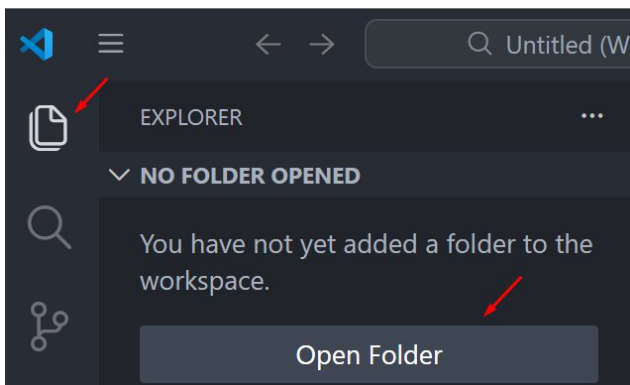
Практическое задание 2

1. Установите **минимум 2 разные темы**.
2. Протестируйте их.
3. Выберите **одну основную тему**.
4. Удалите все лишние.
5. Создайте в рабочей папке файл **myTheme_FIO.txt** и в нём напишите, почему выбрали именно её.

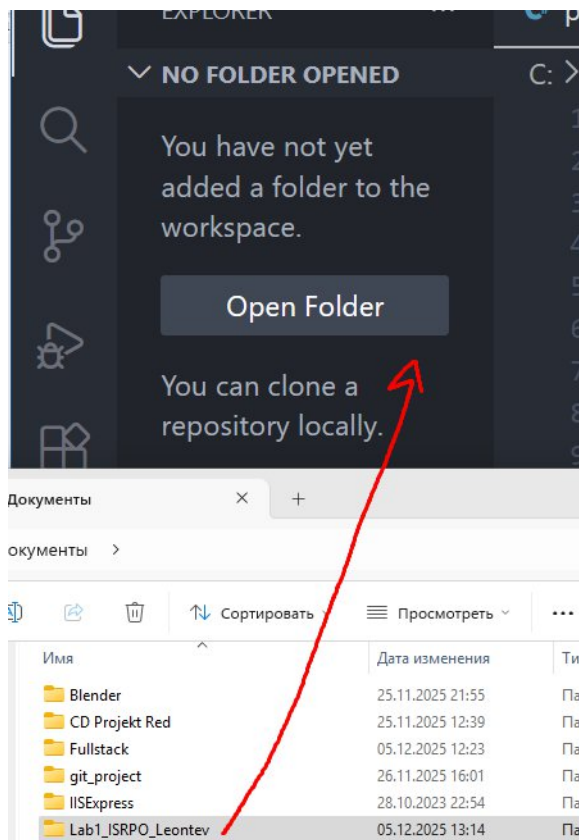
Шаг 3. Добавление проекта в Workspace

Чтобы добавить наш проект мы должны подключить его в так называемый **Workspace (рабочая область)**.

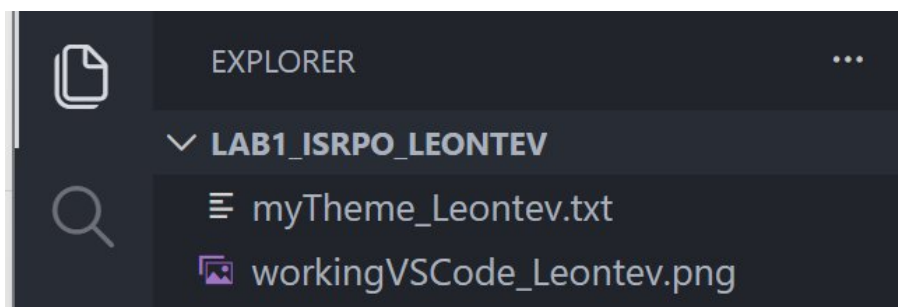
- Откройте **Explorer**
- Нажмите **Open Folder**
- Выберите папку проекта



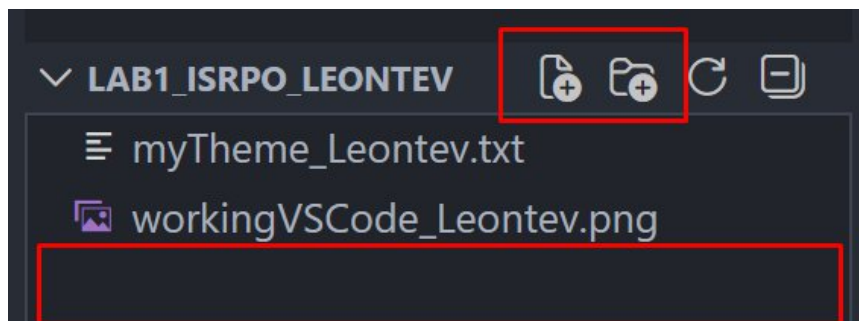
Другой способ заключается в переносе папки в вашу рабочую область.



Мы видим, что наша папка появилась:

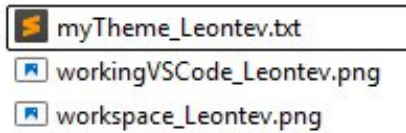


Для того чтобы создать в ней файлы или папки, вы можете или щёлкнуть **ПКМ** по рабочей области или воспользоваться иконками:



Практическое задание 3

Сделайте скриншот открытого проекта в **VS Code** с подключенными проектом и назовите его ***workspace_FIO.png***



Шаг 4. Создание GitHub-репозитория и инициализация Git

4.1. Создание репозитория на GitHub

1. Зайдите на сайт **GitHub** под своей учётной записью.
2. Создайте **новый репозиторий**:
 - о Название: **Lab1-Fullstack_FIO**
 - о Тип: **Public**
 - о Инициализацию с README **НЕ включать**

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

4.2. Настройка терминала Git в VS Code

Для удобства мы будем использовать терминал Git Bash, который автоматически устанавливается вместе с Git.

1. **Откройте терминал в VS Code:**

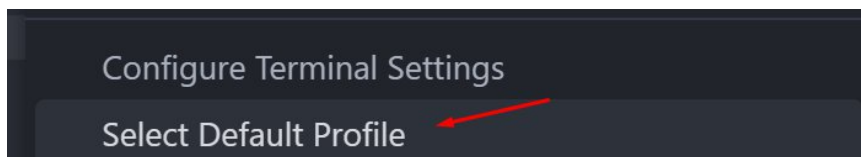
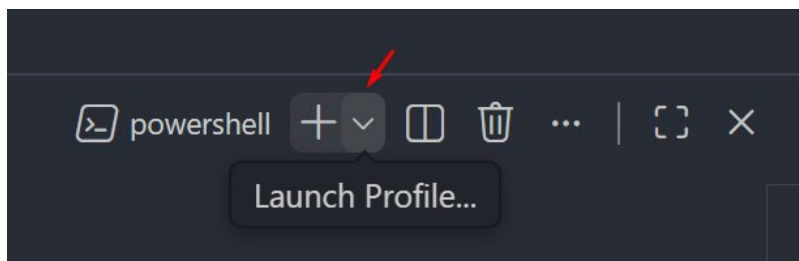
Нажмите сочетание клавиш

Ctrl + ~

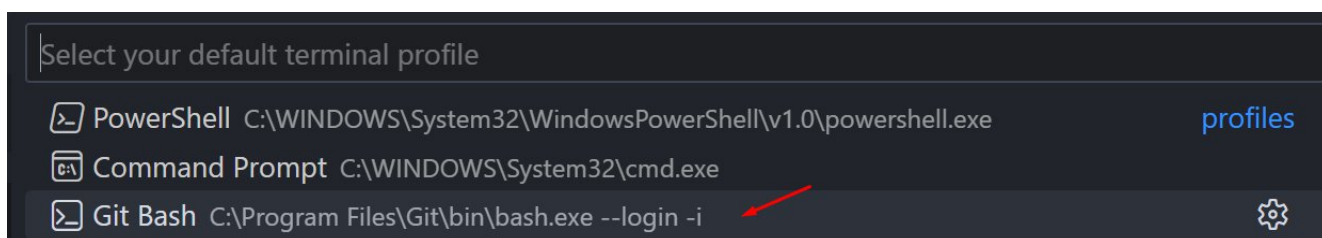
По умолчанию, скорее всего, откроется терминал PowerShell.

Измените профиль терминала по умолчанию на Git Bash:

- Нажмите на стрелку (или иконку шестерёнки) рядом с плюсом в панели терминала и выберите «**Select Default Profile**».



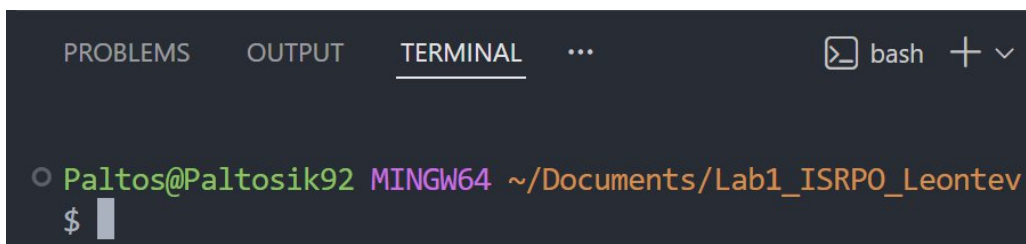
- В появившемся списке выберите **Git Bash**



- Закройте текущий терминал и откройте новый, нажав на +. Теперь вы будете работать в терминале Git Bash:

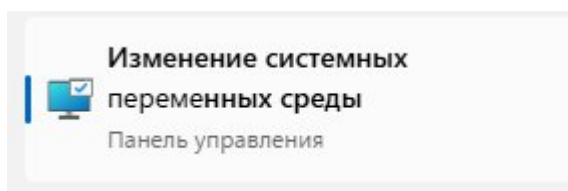


Теперь у вас будет отображаться терминал Git Bash:

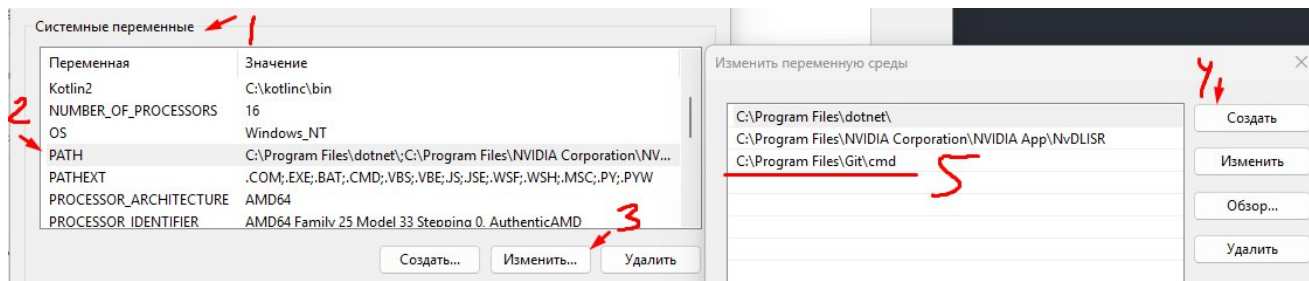


Если **Git Bash** не отображается в списке, добавьте его путь в системные переменные:

- В поиске Windows введите «**изменение системных переменных среды**» и откройте это меню.



2. В разделе «Системные переменные» найдите переменную Path.
3. Нажмите «Изменить» и добавьте новый путь: C:\Program Files\Git\cmd (или путь к папке cmd вашей установки Git).
4. Сохраните изменения, нажав «ОК» во всех открытых окнах, и перезапустите VS Code.



4.3. Инициализация Git через терминал

1. Инициализируйте Git-репозиторий **в папке вашего проекта**:

git init

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev
$ git init
Initialized empty Git repository in C:/Users/Paltos/Documents
/Lab1_ISRPO_Leontev/.git/
```

2. Привяжите удалённый репозиторий GitHub к вашему локальному проекту. Скопируйте URL вашего репозитория на GitHub:

```
...or push an existing repository from the command line
git remote add origin https://github.com/Paltosik92/Lab1-Fullstack_Leontev.git
git branch -M main
git push -u origin main
```

И выполните команду:

git remote add origin ссылка_на_репозиторий

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ git remote add origin https://github.com/Paltosik92/Lab1-Fullstack_Leontev.git
```

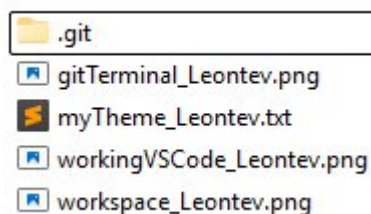
Краткое объяснение команд Git (которые будем использовать постоянно)

- **git init** — создаёт новый локальный репозиторий Git в текущей папке.
- **git add .** — добавляет все изменённые и новые файлы в индекс (staging area) для последующего коммита.

- **git commit -m "сообщение"** — создаёт коммит (сохранённую версию) всех проиндексированных файлов с поясняющим сообщением.
- **git remote add origin <URL>** — связывает локальный репозиторий с удалённым репозиторием на GitHub по указанному адресу.
- **git remote set-url origin <новый_URL>** — изменяет URL удалённого репозитория с псевдонимом origin.
- **git push** — отправляет все локальные коммиты в удалённый репозиторий на GitHub.

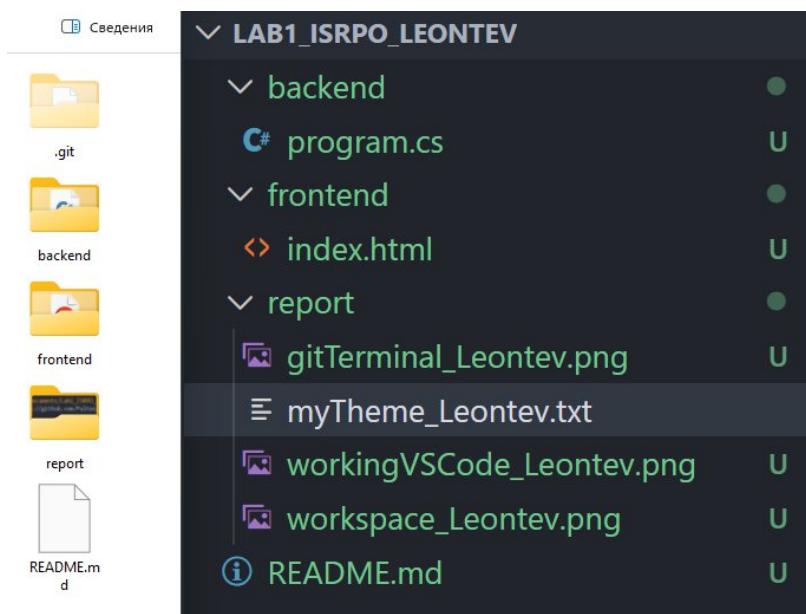
Практическое задание 4

1. Выполните действия из пунктов 4.1–4.3.
2. Сделайте скриншот терминала с выполненными командами **gitTerminal_FIO.png** и сохраните его в папку:



Шаг 5. Создание структуры файлов и папок

1. Внутри проекта создайте файл **program.cs**
2. Создайте следующую структуру:



3. Удалите папку из Workspace:
 - ПКМ → **Remove Folder from Workspace**
4. Подключите её обратно через **Open Folder**.

Шаг 6. Первый коммит и отправка на GitHub

После создания структуры проекта выполните следующие команды в терминале:

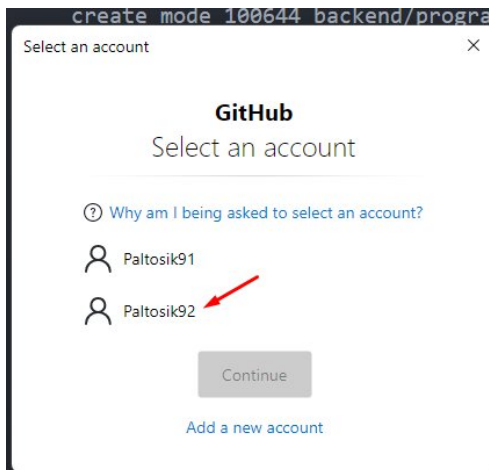
git add .

git status

git commit -m "Initial project structure"

git push -u origin master

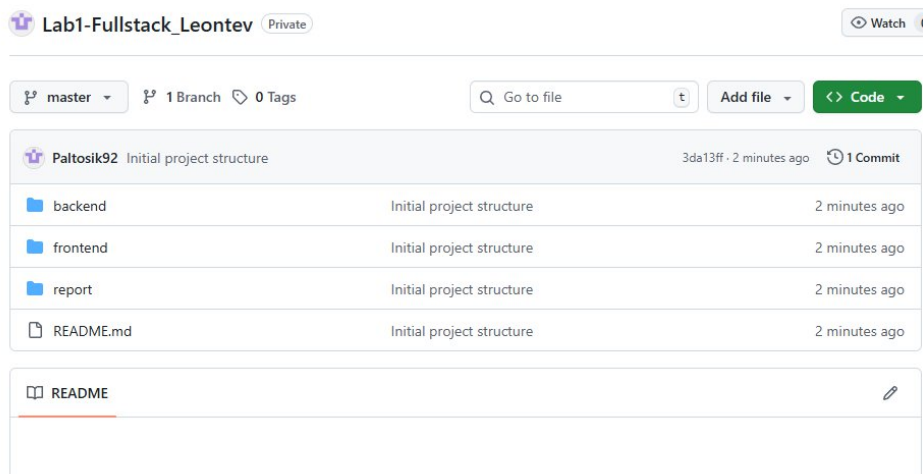
После выполнения команды **git push** у вас должно появиться окно с выбором учётной записи **GitHub** для подтверждения входа.



На этом этапе могут возникнуть проблемы с авторизацией, способы их решения описаны ниже.

Практическое задание 5

1. Убедитесь, что файлы появились на сайте.
2. В папке **report** сохраните скриншот выполненных программ в репозитории с именем: **gitRepo_FIO.png**
3. Сделайте скриншот отправленным в репозиторий в папку **report** с именем: **gitHubRepo_FIO.png**



Решение проблем при push

Если вы работаете за общими компьютерами, возможны проблемы из-за разных пользователей.

1. Проверка текущего пользователя Git

Выполните:

git config user.name

git config user.email

Значения должны **совпадать** с данными вашей учётной записи GitHub.

При необходимости можно задать их вручную:

git config user.name "ваш_ник"

git config user.email "ваша_почта"

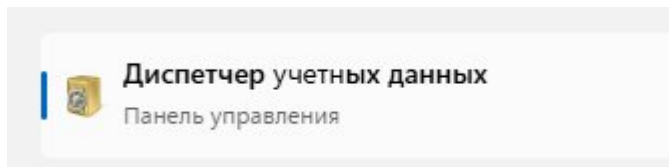
2. Вход в GitHub через браузер

Перед выполнением `git push` вы должны **выполнить вход в GitHub в браузере по умолчанию**, так как при отправке данных появится окно подтверждения авторизации.

3. Очистка старых учётных данных Windows

1. Введите в поиске Windows:

Диспетчер учётных данных



2. Перейдите во вкладку **Учётные данные Windows**

Администрирование учетных данных

Просмотр и удаление сохраненных сведений о входе для веб-сайтов, подключенных приложений и сетей.



Учетные данные для Интернета



Учетные данные Windows

3. Удалите все сохранённые записи, связанные с **GitHub**

4. Проверка привязки удалённого репозитория

Выполните команду:

git remote -v

Вы увидите список удалённых репозиториев:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ git remote -v
origin  https://github.com/Paltosik92/Lab1-Fullstack_Leontev.git (fetch)
origin  https://github.com/Paltosik92/Lab1-Fullstack_Leontev.git (push)
```

- **origin** — стандартное имя главного удалённого репозитория
- **fetch** — адрес для получения данных
- **push** — адрес для отправки данных

Если у вас привязан **не тот репозиторий**, выполните перепривязку:

git remote set-url origin <новый_URL>

5. Удаление папки .git

Вы можете удалить папку **.git**, которая содержит **всю историю проекта**, конфигурационные файлы, объекты, ветки и метаданные, необходимые для работы системы контроля версий в текущем проекте и ввести заново все команды:



Шаг 7. Знакомство со встроенным терминалом

1. В терминала проверьте текущую папку:

pwd

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ pwd
/c/Users/Paltos/Documents/Lab1_ISRPO_Leontev
```

2. Создайте папку:

mkdir terminal_test

3. Выведите все файлы в директории:

ls -lah

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ ls -lah
total 20K
drwxr-xr-x 1 Paltos 197121 0 Dec  6 09:55 ./
drwxr-xr-x 1 Paltos 197121 0 Dec  5 15:46 ../
drwxr-xr-x 1 Paltos 197121 0 Dec  5 15:14 .git/
drwxr-xr-x 1 Paltos 197121 0 Dec  5 15:05 backend/
drwxr-xr-x 1 Paltos 197121 0 Dec  5 15:05 frontend/
-rw-r--r-- 1 Paltos 197121 0 Dec  5 15:05 README.md
drwxr-xr-x 1 Paltos 197121 0 Dec  5 15:16 report/
drwxr-xr-x 1 Paltos 197121 0 Dec  6 09:55 terminal_test/
```

Флаги означают:

- -l — детальная информация
- -h — удобный для чтения размер
- -a — показ скрытых файлов

4. Сделайте скриншот, выполненной работы с именем ***bashTerminal_FIO.png*** в папке **report**.

5. Проверьте состояние Git:

git status

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  report/bashTerminal_Leontev.png
  report/gitHubRepo_Leontev.png
  report/gitRepo_Leontev.png

nothing added to commit but untracked files present (use "git add" to track)
```

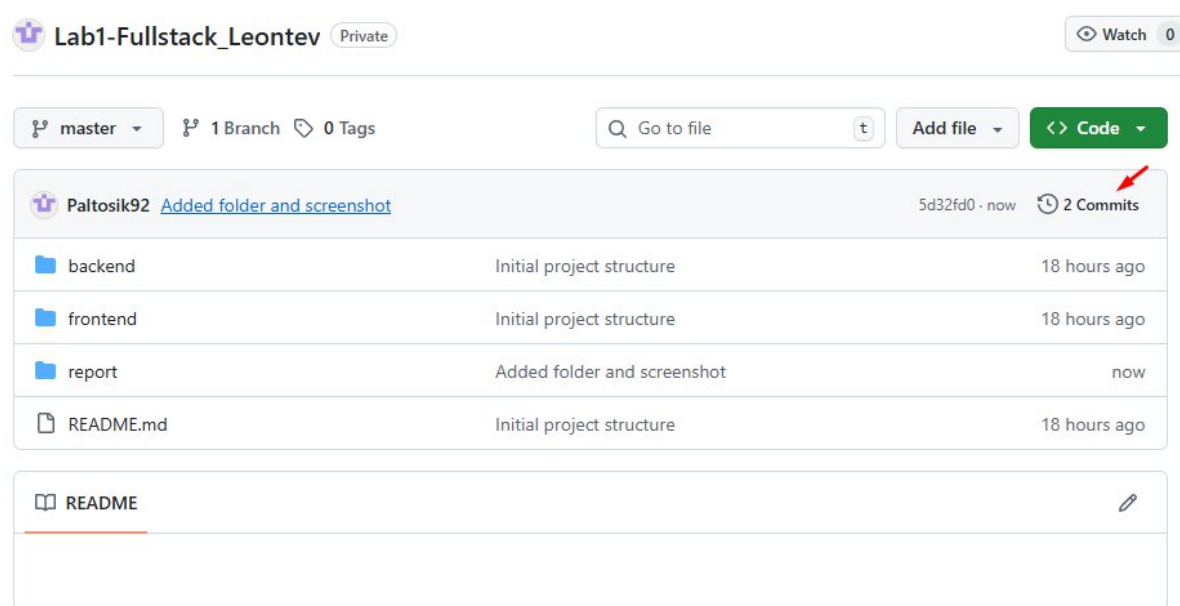
Вы увидите, что появились **неотслеживаемые файлы**.

6. Зафиксируйте изменения:

git add .

git commit -m "Added folder and screenshot"

git push



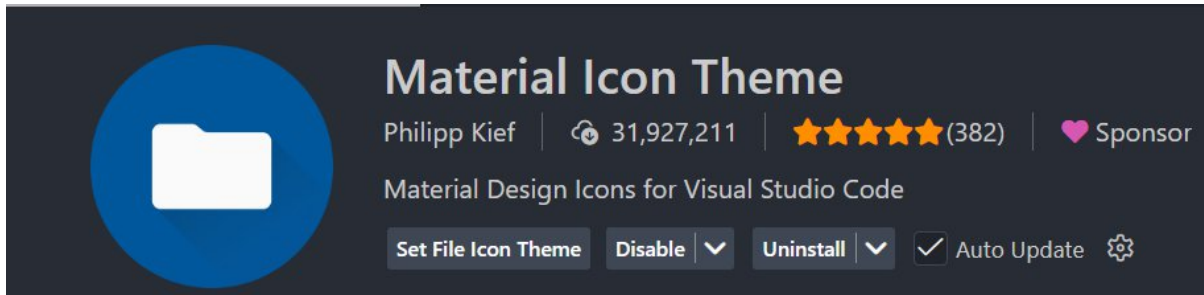
7. Убедитесь, что в вашем **GitHub**-репозитории появился **второй коммит**.

Шаг 8. Изменение иконок файлов

Для более наглядного отображения файлов и папок установим расширение **Material Icon Theme**.

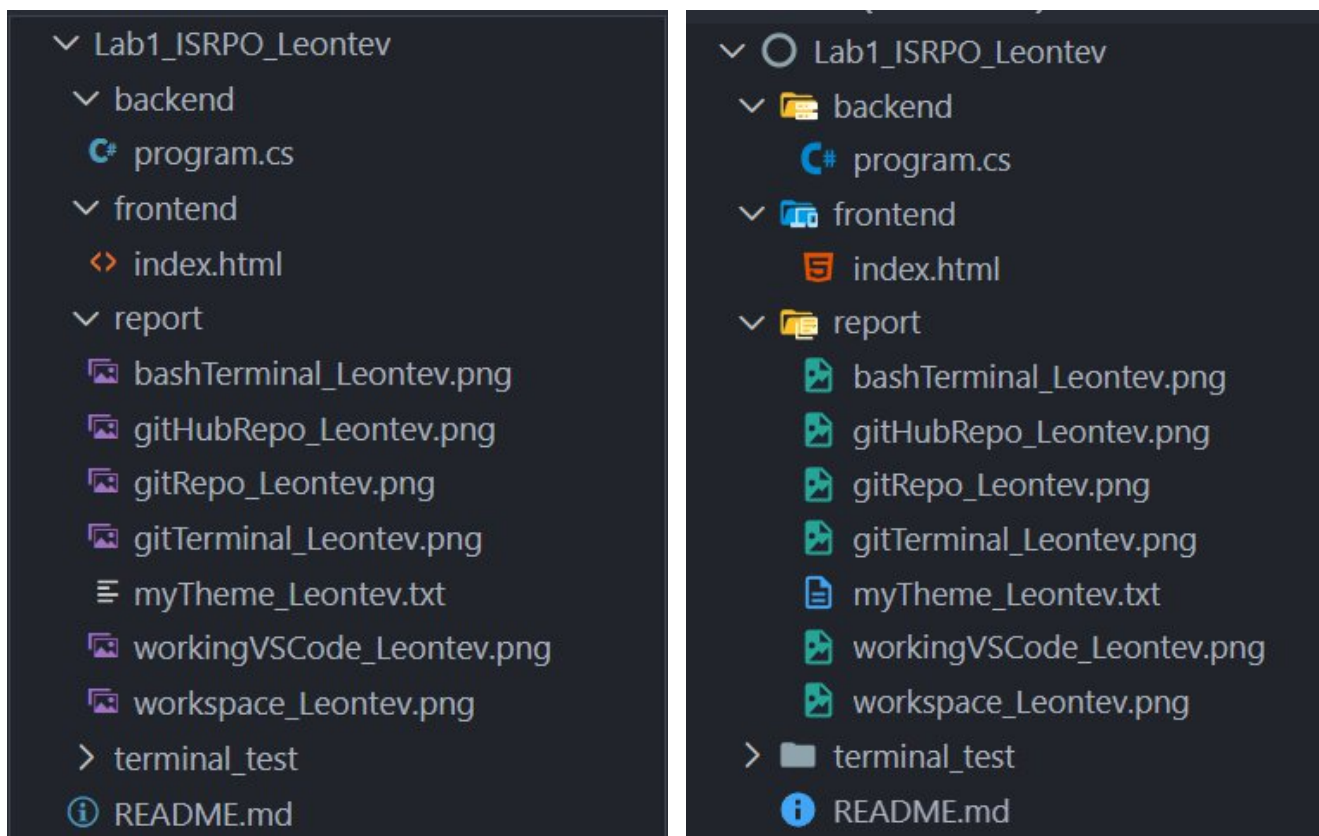
1. Перейдите во вкладку **Extensions**.
2. Введите в поиске:

Material Icon Theme



3. Установите расширение.

После установки иконки файлов и папок станут более крупными и наглядными.



Шаг 9. Изменение размера шрифта

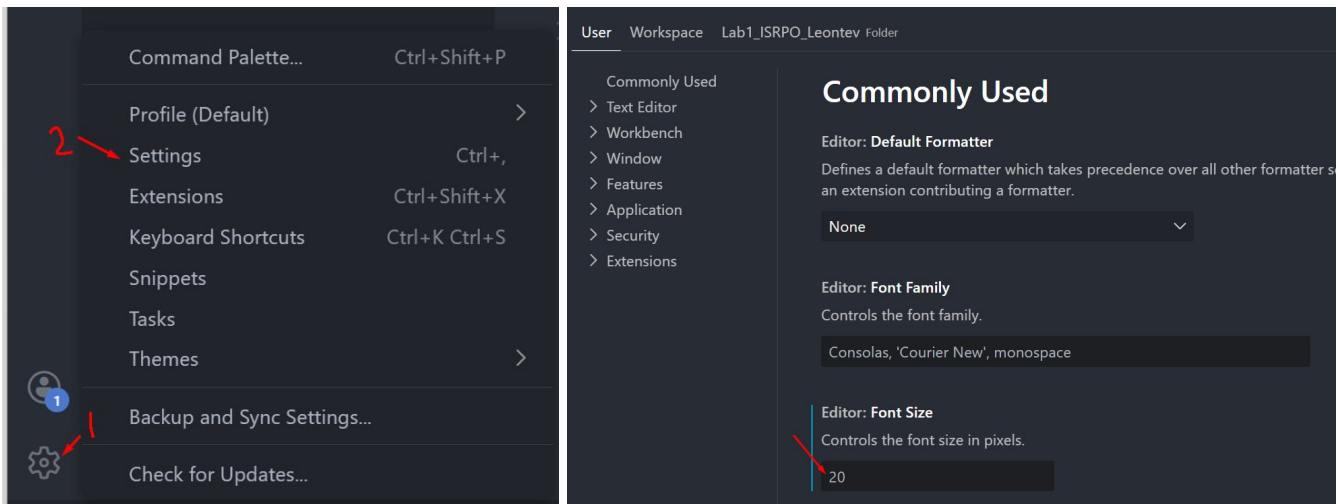
1. Откройте настройки:

Manage → Settings

2. В начале настроек найдите параметр:

Font Size

3. Установите значение:



Практическое задание 6

1. Сделайте и сохраните скриншот изменения шрифта с именем **fontSize_FIO.png** в папке **report**.

2. Зафиксируйте изменения:

git add .

git commit -m "Added font size screenshot"

git push

3. Убедитесь, что в вашем GitHub-репозитории появился **третий коммит**.

Шаг 10. Настройка через конфигурационный файл (settings.json)

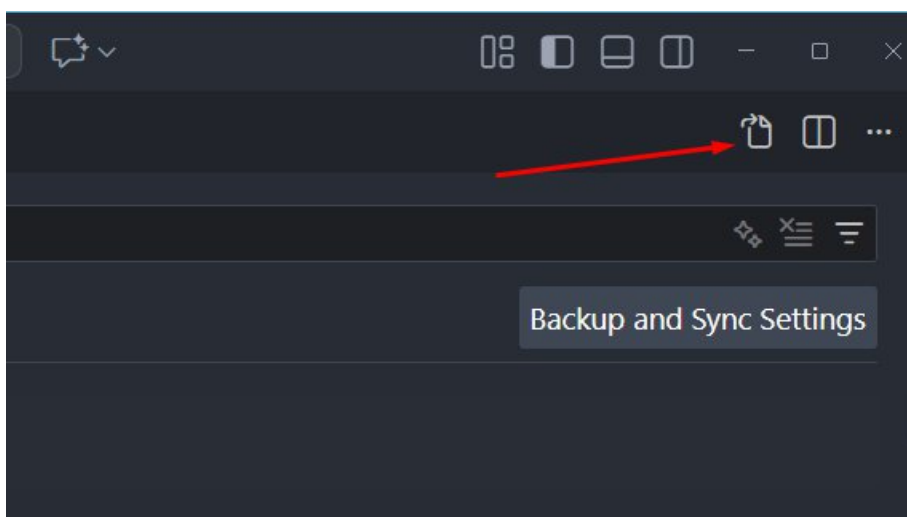
Настройки можно менять не только через меню, но и напрямую через конфигурационный JSON-файл.

1. Откройте:

Manage → Settings

2. В правом верхнем углу нажмите кнопку:

Open Settings (JSON)



3. Перед вами откроется файл **settings.json**.

В качестве примера я выложил свои настройки в **файле** с лабораторной работой.

Практическое задание 7

Ваша задача:

- Изучить содержимое файла **settings.json**.
- Внести свои изменения **или** оставить предложенные настройки.

1. Сделайте и сохраните скриншот изменения JSON-файла с именем **settingsJSON_FIO.png** в папке **report**.

2. Зафиксируйте изменения:

git add .

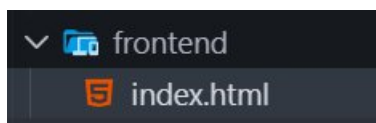
git commit -m "Change settings JSON"

git push

3. Убедитесь, что в вашем GitHub-репозитории появился **четвёртый коммит**.

Шаг 11. Наполнение frontend: создание index.html и просмотр в браузере

1. Перейдите в папку **frontend** и откройте файл **index.html**



2. Вставьте базовую HTML-страницу:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Пример HTML-страницы</title>
</head>
<body>
  <h1>Пример базовой HTML-страницы</h1>
  <h2>Изучение веб-технологий</h2>
  <p>Эта страница демонстрирует базовые элементы HTML. HTML - это язык разметки, который определяет структуру веб-страницы.</p>
  <h3>Технологии Fullstack-разработки:</h3>
  <ul>
    <li>HTML - структура страницы</li>
    <li>CSS - стили и оформление</li>
    <li>JavaScript - клиентская логика</li>
    <li>C# - серверная логика</li>
  </ul>
</body>
</html>
```

3. Сохраните файл.

Просмотр в браузере

1. Перенесите файл **index.html** в браузер.
2. Убедитесь, что страница отображается корректно.

Практическое задание 8

1. Заполните файл **index.html** собственным текстом с **вашими данными**:

- о **ФИО**
- о **Группа**
- о **Название дисциплины**

```
<p>Иванов Иван Иванович</p>
<p>ИСП-231</p>
<p>ИСРПО</p>
```

2. Сделайте скриншот страницы в **браузере**:

frontendPreview_FIO.png

3. Сохраните скриншот в папке:

report

4. Зафиксируйте изменения в терминале:

git status

git add .

git commit -m "Added first frontend page"

git push

5. Убедитесь, что в GitHub появился **новый коммит**.

Шаг 12. Наполнение backend: редактирование файла program.cs

1. В терминале перейдите в папку **backend**

cd backend

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ ls
backend/  frontend/  README.md  report/  terminal_test/

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ cd backend

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev/backend (master)
$ █
```

2. Создадим консольное приложение:

dotnet new console

На этом этапе система выдаст ошибку. Это связано с тем, что файл Program.cs у нас уже существует.

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev/backend (master)
$ dotnet new console
При создании этого шаблона будут внесены изменения в существующие файлы:
  Перезаписать   ./backend.csproj
  Перезаписать   ./Program.cs

Чтобы создать шаблон в любом случае, выполните команду с параметром "--force":
  dotnet new console --force

Дополнительные сведения о коде выхода см. на странице https://aka.ms/templating-exit-codes#73
```

Так как мы в нём ещё ничего не писали, воспользуемся подсказкой и пересоздадим проект с принудительной перезаписью:

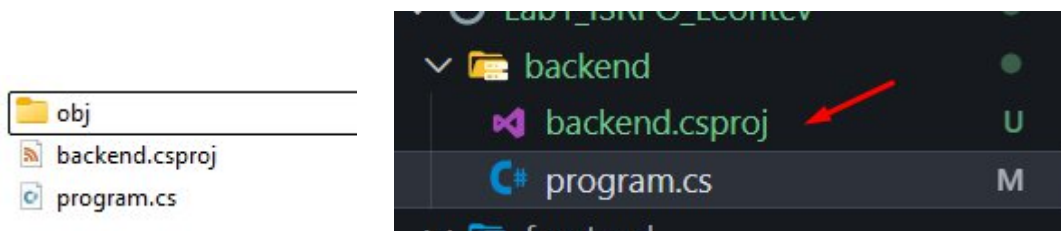
dotnet new console --force

3. После этого обратите внимание:

- В папке появился файл проекта **.csproj**
- В проводнике также появилась папка **obj**

Папка obj содержит служебные и кэшируемые файлы.

Её отображение в VS Code у нас **отключено через настройки**.



Практическое задание 9

1. Реализуйте простую консольную программу:

- **Вывод приветствия**
- **Вывод ФИО**
- **Вывод текущей даты и времени (DateTime.Now)**

2. Запустите программу в терминале командой:

dotnet run


```
Paltos@Paltosik92 MINGW64 ~/
$ dotnet run
Hello, World!
Меня зовут Леонтьев Д.А.
06.12.2025 12:48:23
```

3. Сделайте скриншот **кода** в VS Code:

backendCode_FIO.png

4. Сохраните в папке:

report

5. Вернитесь в корневую папку проекта:

cd ..

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev/backend (master)
$ cd ..

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$
```

6. Зафиксируйте изменения:

git add .

git commit -m "Added basic backend program"

git push

7. Если выполнить команду:

git status

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   backend/backend.csproj
    new file:   backend/bin/Debug/net10.0/backend.deps.json
    new file:   backend/bin/Debug/net10.0/backend.dll
    new file:   backend/bin/Debug/net10.0/backend.exe
    new file:   backend/bin/Debug/net10.0/backend.pdb
```

Вы увидите, что теперь у нас добавилась папка **obj**, которая содержит кэшируемые файлы и в репозиторий нам не нужна. Позже мы научимся убирать такие папки через файл **.gitignore**.

8. Убедитесь, что в вашем репозитории появился **ещё один коммит**.

Шаг 13. Работа с файлами через терминал (bash-практика)

Все действия выполняются **ТОЛЬКО** через терминал.

1. Очистите вывод в терминале:

clear

2. Создайте папки:

mkdir logs

mkdir temp

3. Выведите структуру:

ls

4. Создайте файл:

touch logs/app.log

5. Скопируйте файл:

cp logs/app.log temp/app_copy.log

5. Переименуйте файл:

mv temp/app_copy.log temp/app_backup.log

6. Выведите полную структуру:

ls -lah

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
• $ mkdir logs

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
• $ mkdir temp

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
• $ ls
backend/  frontend/  logs/  README.md  report/  temp/  terminal_test/

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
• $ touch logs/app.log

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
• $ cp logs/app.log temp/app_copy.log

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
• $ mv temp/app_copy.log temp/app_backup.log

Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
• $ ls -lah
```

Практическое задание 10

1. Сделайте скриншот терминала:

terminalFiles_FIO.png

2. Сохраните его в папке:

report

4. Зафиксируйте изменения:

git add .

git commit -m "Terminal file operations"

git push

5. Убедитесь, что появился **ещё один коммит**.

Шаг 14. Горячие клавиши VS Code (ускорение работы)

Опробуйте в любом файле:

Shift + Alt + ↓ — дублирование строки

Alt + ↑ — переместить строку

Alt + ЛКМ — множественные курсоры

Ctrl + / — комментирование

Ctrl + . — быстрые действия

Ctrl + Shift + F — Поиск по проекту

Ctrl + B — скрыть/показать боковую панель

Ctrl + J — скрыть/показать терминал

Ctrl + P — быстрый поиск файлов

F2 — переименование переменной во всем проекте

Практическое задание 11

1. Используя горячие клавиши:

- o Создайте дубликат строки в **index.html**
- o Закомментируйте одну строку

2. Сделайте скриншот:

hotkeys_FIO.png

3. Сохраните в папке:

report

4. Зафиксируйте изменения:

git add .

git commit -m "Used VS Code hotkeys"

git push

Заключение

В ходе лабораторной работы вы научились:

- Настраивать VS Code
- Работать с Git через терминал
- Управлять ветками и коммитами
- Писать базовый frontend и backend
- Использовать горячие клавиши и терминал

Самостоятельные задания

В некоторых заданиях присутствует материал, который не разбирался на паре, и вам нужно будет *загуглить* нужную информацию.

Задание 1. Расширение frontend

В папке **frontend**:

1. Создайте новый файл:

contacts.html

2. В файле должно быть:

- Заголовок: **Контакты**
- ФИО и Группа
- Email (любой)
- Телефон (любой)
- Кнопка возврата на index.html

3. В файле **index.html**:

- Добавьте ссылку на **contacts.html**

➤ Коммиты:

- **Коммит** — создание contacts.html
- **Коммит** — наполнение contacts.html
- **Коммит** — редактирование index.html

Задание 2. Улучшение backend

В файле **backend/program.cs** добавить:

1. Ввод имени пользователя с клавиатуры
2. Персональное приветствие:
Привет, Иван!
3. Вывод текущего дня недели

4. Запрос:

Хотите продолжить? (y/n)

И обработку ответа

➤ Коммиты:

- Коммит — ввод имени
- Коммит — день недели
- Коммит — логика продолжения

Задание 3. Практика терминала

В корне проекта через терминал:

1. Создать папку:

terminal_advanced

2. В ней создать:

file1.txt

file2.txt

file3.txt

3. Объединить их в один файл:

all.txt

➤ Коммиты:

- **Коммит** — создание файлов
- **Коммит** — объединение

Задание 4. Работа с историей Git

1. Выполните:

git log

2. Найдите:

- самый первый коммит
- последний коммит

3. Сделайте скриншот с историей:

gitLog_FIO.png

4. Сохраните в папку **report**

5. Отправьте в удалённый репозиторий.

➤ Коммит:

- **Коммит** — добавлен лог Git